INSERTION AND DELETION OF AN ELEMENT IN BINARY SEARCH TREE

```
#include <iostream>

using namespace std;


struct Node

{

   int data;

   Node* left;

   Node* right;

};


// Create new node

Node* createNode(int value)

{

   Node* newNode = new Node();

   newNode->data = value;

   newNode->left = newNode->right = NULL;

   return newNode;

}


// Insert element in BST

Node* insert(Node* root, int value)

{

   if(root == NULL)

      return createNode(value);


   if(value < root->data)

      root->left = insert(root->left, value);
```

```
    else
        root->right = insert(root->right, value);

    return root;
}

// Find minimum value node
Node* findMin(Node* root)
{
    while(root->left != NULL)
        root = root->left;
    return root;
}

// Delete element from BST
Node* deleteNode(Node* root, int key)
{
    if(root == NULL)
        return NULL;

    if(key < root->data)
        root->left = deleteNode(root->left, key);

    else if(key > root->data)
        root->right = deleteNode(root->right, key);

    else
    {
```

```
    // Case 1: No child
    if(root->left == NULL && root->right == NULL)
    {
        delete root;
        return NULL;
    }

    // Case 2: One child
    else if(root->left == NULL)
    {
        Node* temp = root->right;
        delete root;
        return temp;
    }

    else if(root->right == NULL)
    {
        Node* temp = root->left;
        delete root;
        return temp;
    }

    // Case 3: Two children
    Node* temp = findMin(root->right);
    root->data = temp->data;
    root->right = deleteNode(root->right, temp->data);
}
```

```
    return root;
}

// Inorder traversal
void inorder(Node* root)
{
    if(root != NULL)
    {
        inorder(root->left);
        cout << root->data << " ";
        inorder(root->right);
    }
}

int main()
{
    Node* root = NULL;

    root = insert(root, 50);
    insert(root, 30);
    insert(root, 20);
    insert(root, 40);
    insert(root, 70);
    insert(root, 60);
    insert(root, 80);

    cout << "Inorder before deletion: ";
    inorder(root);
```

```
    root = deleteNode(root, 20);

    cout << "\nInorder after deletion: ";
    inorder(root);

    return 0;
}
```

OUTPUT :

```
Output
Inorder before deletion: 20 30 40 50 60 70 80
Inorder after deletion: 30 40 50 60 70 80

=== Code Execution Successful ===
```